Semantic declarations

`bxor`

Kind: `definition`.
Parameters: `(a : Bool) (b : Bool)`.
Type: `Bool`.
Definition: `(if a then (!b) else b)`.
Axiom policy: `none`.

`rule`

Kind: `definition`.
Parameters: `(a : Bool) (b : Bool) (c : Bool)`.
Type: `Bool`.
Definition: `bxor (a) ((b || c))`.
Axiom policy: `none`.

`rule90`

Kind: `definition`.
Parameters: `(a : Bool) (b : Bool) (c : Bool)`.
Type: `Bool`.
Definition: `bxor (a) (c)`.
Axiom policy: `none`.

`rule150`

Kind: `definition`.
Parameters: `(a : Bool) (b : Bool) (c : Bool)`.
Type: `Bool`.
Definition: `bxor (bxor (a) (b)) (c)`.
Axiom policy: `none`.

`rule210`

Kind: `definition`.
Parameters: `(a : Bool) (b : Bool) (c : Bool)`.
Type: `Bool`.
Definition: `bxor (a) (((!b) && c))`.
Axiom policy: `none`.

`barrier_plane`

Kind: `theorem`.
Statement: `(forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (rule (a) (b) (c) = bxor (bxor (rule90 (a) (b) (c)) (rule150 (a) (b) (c))) (rule210 (a) (b) (c))))))`.
Axiom policy: `none`.

`left_permutive`

Kind: `theorem`.
Statement: `(forall (b : Bool), (forall (c : Bool), (rule (true) (b) (c) = (!rule (false) (b) (c)))))`.
Axiom policy: `none`.

`cell`

Kind: `definition`.
Parameters: `(t : Nat) (p : Int)`.
Type: `Bool`.
Definition: `(match t with | Nat.zero => ((!(LexLeanCollections.lessThan (p) ((0 : Int)) : Bool)) && (!(LexLeanCollections.lessThan ((0 : Int)) (p) : Bool))) | Nat.succ s => rule (cell (s) ((LexLeanRuntime.subtract (p) ((1 : Int)) : Int))) (cell (s) (p)) (cell (s) ((LexLeanRuntime.subtract (p) ((-1 : Int)) : Int))))`.
Axiom policy: `exact [propext]`.

`center`

Kind: `definition`.
Parameters: `(t : Nat)`.
Type: `Bool`.
Definition: `cell (t) ((0 : Int))`.
Axiom policy: `exact [propext]`.

`center_prefix`

Kind: `theorem`.
Statement: `((center (0) = true) /\ ((center (1) = true) /\ ((center (2) = false) /\ (center (3) = true))))`.
Axiom policy: `exact [propext]`.

leftOf

Kind: definition.
Parameters: (p : Int).
Type: Int.
Definition: (LexLeanRuntime.subtract (p) ((1 : Int)) : Int).
Axiom policy: none.

rightOf

Kind: definition.
Parameters: (p : Int).
Type: Int.
Definition: (LexLeanRuntime.subtract (p) ((-1 : Int)) : Int).
Axiom policy: none.

cell_succ

Kind: theorem.
Parameters: (t : Nat) (p : Int).
Statement: (cell (Nat.succ (t)) (p) = rule (cell (t) (leftOf (p))) (cell (t) (p)) (cell (t) (rightOf (p)))).
Axiom policy: exact [propext].

rule_black_black

Kind: theorem.
Statement: (forall (c : Bool), (rule (true) (true) (c) = false)).
Axiom policy: none.

rule_white_white

Kind: theorem.
Statement: (forall (a : Bool), (rule (a) (false) (false) = a)).
Axiom policy: none.

rule_left_edge_of_white_pair

Kind: theorem.
Statement: (forall (c : Bool), (rule (false) (false) (c) = c)).
Axiom policy: none.

rule_reset

Kind: theorem.
Statement: (forall (a : Bool), (forall (c : Bool), (rule (a) (true) (c) = (!a)))).
Axiom policy: none.

ldiag

Kind: definition.
Parameters: (j : Nat) (t : Nat).
Type: Bool.
Definition: (match t with | Nat.zero => (match j with | Nat.zero => true | Nat.succ _ => false) | Nat.succ s => (match j with | Nat.zero => true | Nat.succ i => (match i with | Nat.zero => true | Nat.succ k => bxor (ldiag (k) (s)) ((ldiag (Nat.succ (k)) (s) || ldiag (Nat.succ (Nat.succ (k))) (s)))))).
Axiom policy: none.

lcenter

Kind: definition.
Parameters: (t : Nat).
Type: Bool.
Definition: ldiag (t) (t).
Axiom policy: none.

ldiag_step

Kind: theorem.
Parameters: (k : Nat) (s : Nat).
Statement: (ldiag (Nat.succ (Nat.succ (k))) (Nat.succ (s)) = bxor (ldiag (k) (s)) ((ldiag (Nat.succ (k)) (s) || ldiag (Nat.succ (Nat.succ (k))) (s)))).
Axiom policy: none.

ldiag_zero

Kind: theorem.
Parameters: (s : Nat).
Statement: (ldiag (0) (Nat.succ (s)) = true).
Axiom policy: none.

ldiag_one

Kind: `theorem`.
Parameters: `(s : Nat)`.
Statement: `(ldiag (1) (Nat.succ (s)) = true)`.
Axiom policy: `none`.

rdiag

Kind: `definition`.
Parameters: `(j : Nat) (t : Nat)`.
Type: `Bool`.
Definition: `(match t with | Nat.zero => (match j with | Nat.zero => true | Nat.succ _ => false) | Nat.succ s => (match j with | Nat.zero => true | Nat.succ i => (match i with | Nat.zero => (!rdiag (Nat.succ (i)) (s)) | Nat.succ k => bxor (rdiag (Nat.succ (Nat.succ (k))) (s)) ((rdiag (Nat.succ (k)) (s) || rdiag (k) (s))))))`.
Axiom policy: `none`.

rcenter

Kind: `definition`.
Parameters: `(t : Nat)`.
Type: `Bool`.
Definition: `rdiag (t) (t)`.
Axiom policy: `none`.

rdiag_step

Kind: `theorem`.
Parameters: `(k : Nat) (s : Nat)`.
Statement: `(rdiag (Nat.succ (Nat.succ (k))) (Nat.succ (s)) = bxor (rdiag (Nat.succ (Nat.succ (k))) (s)) ((rdiag (Nat.succ (k)) (s) || rdiag (k) (s))))`.
Axiom policy: `none`.

rdiag_zero

Kind: `theorem`.
Parameters: `(s : Nat)`.
Statement: `(rdiag (0) (Nat.succ (s)) = true)`.
Axiom policy: `none`.

rdiag_one

Kind: theorem.
Parameters: (s : Nat).
Statement: (rdiag (1) (Nat.succ (s)) = (!rdiag (1) (s))).
Axiom policy: none.

rule60

Kind: definition.
Parameters: (a : Bool) (b : Bool) (c : Bool).
Type: Bool.
Definition: bxor (a) (b).
Axiom policy: none.

rule15

Kind: definition.
Parameters: (a : Bool) (b : Bool) (c : Bool).
Type: Bool.
Definition: (!a).
Axiom policy: none.

split150

Kind: theorem.
Statement: (forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (rule (a) (b) (c) = bxor (rule150 (a) (b) (c)) ((b && c)))))).
Axiom policy: none.

split90

Kind: theorem.
Statement: (forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (rule (a) (b) (c) = bxor (rule90 (a) (b) (c)) ((b && (!c))))))).
Axiom policy: none.

split60

Kind: theorem.
Statement: (forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (rule (a) (b) (c) = bxor (rule60 (a) (b) (c)) (((!b) && c)))))).

Axiom policy: none.

split15

Kind: theorem.
Statement: (forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (rule (a) (b) (c) = bxor (rule15 (a) (b) (c)) (((!b) && (!c))))))).
Axiom policy: none.

carries_cover

Kind: theorem.
Statement: (forall (b : Bool), (forall (c : Bool), ((((b && c) || (b && (!c))) || (((!b) && c) || ((!b) && (!c)))) = true))).
Axiom policy: none.

carries_orthogonal

Kind: theorem.
Statement: (forall (b : Bool), (forall (c : Bool), (((((b && c) && (b && (!c))) = false) /\ ((((b && c) && ((!b) && c)) = false) /\ (((b && c) && ((!b) && (!c))) = false))) /\ (((((b && (!c)) && ((!b) && c)) = false) /\ (((b && (!c)) && ((!b) && (!c))) = false)) /\ ((((!b) && c) && ((!b) && (!c))) = false))))).
Axiom policy: none.

aff

Kind: definition.
Parameters: (u0 : Bool) (u1 : Bool) (u2 : Bool) (u3 : Bool) (a : Bool) (b : Bool) (c : Bool).
Type: Bool.
Definition: bxor (u0) (bxor ((u1 && a)) (bxor ((u2 && b)) ((u3 && c)))).
Axiom policy: none.

miss

Kind: definition.
Parameters: (u0 : Bool) (u1 : Bool) (u2 : Bool) (u3 : Bool) (a : Bool) (b : Bool) (c : Bool).
Type: Nat.

Definition: `(if bxor (rule (a) (b) (c)) (aff (u0) (u1) (u2) (u3) (a) (b) (c)) then 1 else 0)`.
Axiom policy: `none`.

dist30

Kind: `definition`.
Parameters: `(u0 : Bool) (u1 : Bool) (u2 : Bool) (u3 : Bool)`.
Type: `Nat`.
Definition: `(((miss (u0) (u1) (u2) (u3) (false) (false) (false) + miss (u0) (u1) (u2) (u3) (false) (false) (true)) + (miss (u0) (u1) (u2) (u3) (false) (true) (false) + miss (u0) (u1) (u2) (u3) (false) (true) (true))) + ((miss (u0) (u1) (u2) (u3) (true) (false) (false) + miss (u0) (u1) (u2) (u3) (true) (false) (true)) + (miss (u0) (u1) (u2) (u3) (true) (true) (false) + miss (u0) (u1) (u2) (u3) (true) (true) (true))))`.
Axiom policy: `none`.

deep_hole

Kind: `theorem`.
Statement: `(forall (u0 : Bool), (forall (u1 : Bool), (forall (u2 : Bool), (forall (u3 : Bool), (2 <= dist30 (u0) (u1) (u2) (u3))))))`.
Axiom policy: `none`.

nearest_affine

Kind: `theorem`.
Statement: `(((dist30 (false) (true) (true) (true) = 2) /\ (dist30 (false) (true) (false) (true) = 2)) /\ ((dist30 (false) (true) (true) (false) = 2) /\ (dist30 (true) (true) (false) (false) = 2)))`.
Axiom policy: `none`.

carry60

Kind: `definition`.
Parameters: `(s : Nat) (p : Int)`.
Type: `Bool`.
Definition: `((!cell (s) (p)) && cell (s) ((LexLeanRuntime.subtract (p) ((-1 : Int)) : Int)))`.
Axiom policy: `exact [propext]`.

`window60`

Kind: `definition`.
Parameters: `(t0 : Nat) (n : Nat) (p : Int)`.
Type: `Bool`.
Definition: `(match n with | Nat.zero => false | Nat.succ m => bxor (bxor (window60 (t0) (m) ((LexLeanRuntime.subtract (p) ((1 : Int)) : Int))) (window60 (t0) (m) (p))) (carry60 ((t0 + m)) (p)))`.
Axiom policy: `exact [propext]`.

`twopow`

Kind: `definition`.
Parameters: `(k : Nat)`.
Type: `Nat`.
Definition: `(match k with | Nat.zero => 1 | Nat.succ j => (twopow (j) + twopow (j)))`.
Axiom policy: `none`.

`window60_step`

Kind: `theorem`.
Parameters: `(t0 : Nat) (m : Nat) (p : Int)`.
Statement: `(window60 (t0) (Nat.succ (m)) (p) = bxor (bxor (window60 (t0) (m) ((LexLeanRuntime.subtract (p) ((1 : Int)) : Int))) (window60 (t0) (m) (p))) (carry60 ((t0 + m)) (p)))`.
Axiom policy: `exact [propext]`.

`twopow_succ`

Kind: `theorem`.
Parameters: `(j : Nat)`.
Statement: `(twopow (Nat.succ (j)) = (twopow (j) + twopow (j)))`.
Axiom policy: `none`.

`displacement`

Kind: `definition`.
Parameters: `(a : Bool) (b : Bool) (c : Bool)`.
Type: `Bool`.
Definition: `bxor (rule (a) (b) (c)) (rule150 (a) (b) (c))`.
Axiom policy: `none`.

`displacement_is_carry`

Kind: `theorem`.
Statement: `(forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (displacement (a) (b) (c) = (b && c))))).`
Axiom policy: `none`.

`frame_direction`

Kind: `theorem`.
Statement: `(forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (displacement ((!a)) (b) (c) = displacement (a) (b) (c))))).`
Axiom policy: `none`.

`affine_left_linear`

Kind: `theorem`.
Statement: `(forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (rule150 ((!a)) (b) (c) = (!rule150 (a) (b) (c)))))).`
Axiom policy: `none`.

`nearest_exactly`

Kind: `theorem`.
Statement: `(forall (u0 : Bool), (forall (u1 : Bool), (forall (u2 : Bool), (forall (u3 : Bool), ((dist30 (u0) (u1) (u2) (u3) = 2) -> (((((!u0) && (u1 && (u2 && u3))) || ((!u0) && (u1 && ((!u2) && u3)))) || (((!u0) && (u1 && (u2 && (!u3)))) || (u0 && (u1 && ((!u2) && (!u3)))))) = true)))))).`
Axiom policy: `none`.

`ldrem`

Kind: `definition`.
Parameters: `(b : Nat) (r : Nat) (t : Nat).`
Type: `Nat`.
Definition: `(match t with | Nat.zero => r | Nat.succ s => (if (LexLeanCollections.less ((ldrem (b) (r) (s) + ldrem (b) (r) (s))) (b) : Bool) then (ldrem (b) (r) (s) + ldrem (b) (r) (s)) else (LexLeanRuntime.subtract ((ldrem (b) (r) (s) + ldrem (b) (r) (s))) (b) : Nat))).`
Axiom policy: `exact [propext].`

lddig

Kind: definition.
Parameters: (b : Nat) (r : Nat) (t : Nat).
Type: Bool.
Definition: (!(LexLeanCollections.lessThan ((ldrem (b) (r) (t) + ldrem (b) (r) (t))) (b) : Bool)).
Axiom policy: exact [propext].

sgn

Kind: definition.
Parameters: (x : Bool).
Type: Int.
Definition: (if x then (-1 : Int) else (1 : Int)).
Axiom policy: none.

iadd

Kind: definition.
Parameters: (x : Int) (y : Int).
Type: Int.
Definition: (LexLeanRuntime.subtract (x) ((LexLeanRuntime.negate (y) : Int)) : Int).
Axiom policy: none.

frame_projection

Kind: theorem.
Statement: (forall (a : Bool), (forall (b : Bool), (forall (c : Bool), (iadd (sgn (rule (a) (b) (c))) (sgn (rule (a) (b) (c))) = iadd (iadd (sgn (rule150 (a) (b) (c))) (sgn (rule90 (a) (b) (c)))) (iadd (sgn (rule60 (a) (b) (c))) (sgn (rule15 (a) (b) (c)))))))).
Axiom policy: none.

dissent

Kind: definition.
Parameters: (a : Bool) (b : Bool) (c : Bool).
Type: Nat.

Definition: (((if bxor (rule (a) (b) (c)) (rule150 (a) (b) (c)) then
1 else 0) + (if bxor (rule (a) (b) (c)) (rule90 (a) (b) (c)) then
1 else 0)) + ((if bxor (rule (a) (b) (c)) (rule60 (a) (b) (c)) then
1 else 0) + (if bxor (rule (a) (b) (c)) (rule15 (a) (b) (c)) then
1 else 0))).
Axiom policy: none.

one_dissent

Kind: theorem.
Statement: (forall (a : Bool), (forall (b : Bool), (forall (c :
Bool), (dissent (a) (b) (c) = 1)))).
Axiom policy: none.

corr

Kind: definition.
Parameters: (u0 : Bool) (u1 : Bool) (u2 : Bool) (u3 : Bool) (v0
: Bool) (v1 : Bool) (v2 : Bool) (v3 : Bool).
Type: Int.
Definition: iadd (iadd (iadd ((LexLeanRuntime.multiply (sgn (aff (u0)
(u1) (u2) (u3) (false) (false) (false))) (sgn (aff (v0) (v1) (v2)
(v3) (false) (false) (false))) : Int)) ((LexLeanRuntime.multiply
(sgn (aff (u0) (u1) (u2) (u3) (false) (false) (true))) (sgn (aff
(v0) (v1) (v2) (v3) (false) (false) (true))) : Int))) (iadd ((LexLeanRuntime.multip
(sgn (aff (u0) (u1) (u2) (u3) (false) (true) (false))) (sgn (aff
(v0) (v1) (v2) (v3) (false) (true) (false))) : Int)) ((LexLeanRuntime.multiply
(sgn (aff (u0) (u1) (u2) (u3) (false) (true) (true))) (sgn (aff (v0)
(v1) (v2) (v3) (false) (true) (true))) : Int)))) (iadd (iadd ((LexLeanRuntime.multi
(sgn (aff (u0) (u1) (u2) (u3) (true) (false) (false))) (sgn (aff
(v0) (v1) (v2) (v3) (true) (false) (false))) : Int)) ((LexLeanRuntime.multiply
(sgn (aff (u0) (u1) (u2) (u3) (true) (false) (true))) (sgn (aff (v0)
(v1) (v2) (v3) (true) (false) (true))) : Int))) (iadd ((LexLeanRuntime.multiply
(sgn (aff (u0) (u1) (u2) (u3) (true) (true) (false))) (sgn (aff (v0)
(v1) (v2) (v3) (true) (true) (false))) : Int)) ((LexLeanRuntime.multiply
(sgn (aff (u0) (u1) (u2) (u3) (true) (true) (true))) (sgn (aff (v0)
(v1) (v2) (v3) (true) (true) (true))) : Int)))).
Axiom policy: none.

`frame_orthogonal`

Kind: `theorem`.
Statement: `(forall (u0 : Bool), (forall (u1 : Bool), (forall (u2 : Bool), (forall (u3 : Bool), (forall (v0 : Bool), (forall (v1 : Bool), (forall (v2 : Bool), (forall (v3 : Bool), (((bxor (u1) (v1) || (bxor (u2) (v2) || bxor (u3) (v3))) = true) -> (corr (u0) (u1) (u2) (u3) (v0) (v1) (v2) (v3) = (0 : Int)))))))))).`
Axiom policy: `none`.

`bxor_reset`

Kind: `theorem`.
Statement: `(forall (a : Bool), (forall (c : Bool), (bxor (a) ((true || c)) = (!a)))).`
Axiom policy: `none`.

`bxor_toggle`

Kind: `theorem`.
Statement: `(forall (a : Bool), (forall (c : Bool), (bxor (a) ((false || c)) = bxor (a) (c)))).`
Axiom policy: `none`.

`rule_toggle`

Kind: `theorem`.
Statement: `(forall (a : Bool), (forall (c : Bool), (rule (a) (false) (c) = bxor (a) (c)))).`
Axiom policy: `none`.

`phi`

Kind: `definition`.
Parameters: `(y : Nat)`.
Type: `Nat`.
Definition: `(Nat.xor (y) ((Nat.lor ((Nat.shiftLeft (y) (1) : Nat)) ((Nat.shiftLeft (y) (2) : Nat)) : Nat)) : Nat).`
Axiom policy: `exact [propext]`.

`carry`

Kind: `definition`.
Parameters: `(y : Nat)`.
Type: `Nat`.
Definition: `(Nat.land (y) ((Nat.shiftLeft (y) (1) : Nat)) : Nat)`.
Axiom policy: `exact [propext]`.

`phiLinear`

Kind: `definition`.
Parameters: `(y : Nat)`.
Type: `Nat`.
Definition: `(Nat.xor ((Nat.xor (y) ((Nat.shiftLeft (y) (1) : Nat)) : Nat)) ((Nat.shiftLeft (y) (2) : Nat)) : Nat)`.
Axiom policy: `exact [propext]`.

`phi_split`

Kind: `theorem`.
Parameters: `(y : Nat)`.
Statement: `(phi (y) = (Nat.xor (phiLinear (y)) ((Nat.shiftLeft (carry (y)) (1) : Nat)) : Nat))`.
Axiom policy: `exact [Quot.sound, propext]`.

`or_witt_split`

Kind: `theorem`.
Parameters: `(a : Nat) (b : Nat)`.
Statement: `((Nat.lor (a) (b) : Nat) = (Nat.xor ((Nat.xor (a) (b) : Nat)) ((Nat.land (a) (b) : Nat)) : Nat))`.
Axiom policy: `exact [Quot.sound, propext]`.

`bit`

Kind: `definition`.
Parameters: `(n : Nat) (k : Nat)`.
Type: `Nat`.
Definition: `(LexLeanRuntime.remainder ((Nat.shiftRight (n) (k) : Nat)) (2) (0) : Nat)`.
Axiom policy: `none`.

`bit_and_law`

Kind: `theorem`.
Parameters: `(a : Nat) (b : Nat) (k : Nat)`.
Statement: `(bit ((Nat.land (a) (b) : Nat)) (k) = (Nat.land (bit (a) (k)) (bit (b) (k)) : Nat))`.
Axiom policy: `exact [Quot.sound, propext]`.

`bit_or_law`

Kind: `theorem`.
Parameters: `(a : Nat) (b : Nat) (k : Nat)`.
Statement: `(bit ((Nat.lor (a) (b) : Nat)) (k) = (Nat.lor (bit (a) (k)) (bit (b) (k)) : Nat))`.
Axiom policy: `exact [Quot.sound, propext]`.

`bit_xor_law`

Kind: `theorem`.
Parameters: `(a : Nat) (b : Nat) (k : Nat)`.
Statement: `(bit ((Nat.xor (a) (b) : Nat)) (k) = (Nat.xor (bit (a) (k)) (bit (b) (k)) : Nat))`.
Axiom policy: `exact [Quot.sound, propext]`.

`bit_shift_right_law`

Kind: `theorem`.
Parameters: `(a : Nat) (m : Nat) (k : Nat)`.
Statement: `(bit ((Nat.shiftRight (a) (m) : Nat)) (k) = bit (a) ((m + k)))`.
Axiom policy: `exact [propext]`.

`bit_shift_left_law`

Kind: `theorem`.
Parameters: `(a : Nat) (m : Nat) (k : Nat)`.
Statement: `(bit ((Nat.shiftLeft (a) (m) : Nat)) ((m + k)) = bit (a) (k))`.
Axiom policy: `exact [Quot.sound, propext]`.

`row`

Kind: `definition`.
Parameters: `(t : Nat)`.
Type: `Nat`.
Definition: `(match t with | Nat.zero => 1 | Nat.succ s => phi (row (s)))`.
Axiom policy: `exact [propext]`.

`rowCenter`

Kind: `definition`.
Parameters: `(t : Nat)`.
Type: `Nat`.
Definition: `bit (row (t)) (t)`.
Axiom policy: `exact [propext]`.

`row_center_prefix`

Kind: `theorem`.
Statement: `((rowCenter (0) = 1) /\ ((rowCenter (1) = 1) /\ ((rowCenter (2) = 0) /\ (rowCenter (3) = 1))))`.
Axiom policy: `exact [propext]`.

`phiLinearPow`

Kind: `definition`.
Parameters: `(n : Nat) (y : Nat)`.
Type: `Nat`.
Definition: `(match n with | Nat.zero => y | Nat.succ m => phiLinear (phiLinearPow (m) (y)))`.
Axiom policy: `exact [propext]`.

`phiLinearPow_step`

Kind: `theorem`.
Parameters: `(m : Nat) (y : Nat)`.
Statement: `(phiLinearPow (Nat.succ (m)) (y) = phiLinear (phiLinearPow (m) (y)))`.
Axiom policy: `exact [propext]`.

phiScale

Kind: definition.
Parameters: (k : Nat) (y : Nat).
Type: Nat.
Definition: (Nat.xor ((Nat.xor (y) ((Nat.shiftLeft (y) (twopow (k)) : Nat)) : Nat)) ((Nat.shiftLeft (y) ((twopow (k) + twopow (k))) : Nat)) : Nat).
Axiom policy: exact [propext].

winWitt

Kind: definition.
Parameters: (t0 : Nat) (n : Nat).
Type: Nat.
Definition: (match n with | Nat.zero => 0 | Nat.succ m => (Nat.xor (phiLinear (winWitt (t0) (m))) ((Nat.shiftLeft (carry (row ((t0 + m)))) (1) : Nat)) : Nat)).
Axiom policy: exact [propext].

winWitt_step

Kind: theorem.
Parameters: (t0 : Nat) (m : Nat).
Statement: (winWitt (t0) (Nat.succ (m)) = (Nat.xor (phiLinear (winWitt (t0) (m))) ((Nat.shiftLeft (carry (row ((t0 + m)))) (1) : Nat)) : Nat)).
Axiom policy: exact [propext].

wittResponse

Kind: definition.
Parameters: (t : Nat) (k : Nat).
Type: Nat.
Definition: bit (winWitt (t) (twopow (k))) ((t + twopow (k))).
Axiom policy: exact [propext].